Recommender System Challenge 2025-2026

Luigi Inguaggiato

Data Exploration

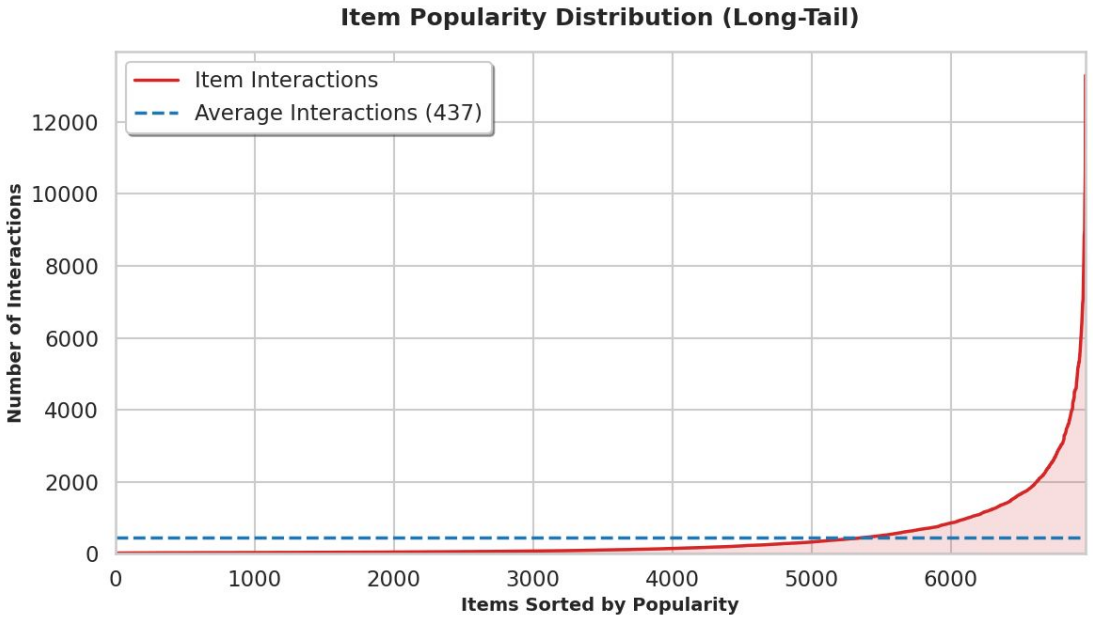
Implicit Ratings URM-Pure Collaborative

Total interactions 3043058

Number of items 6969

Number of users 27095

Sparsity 98.39 %



Initial Attempts

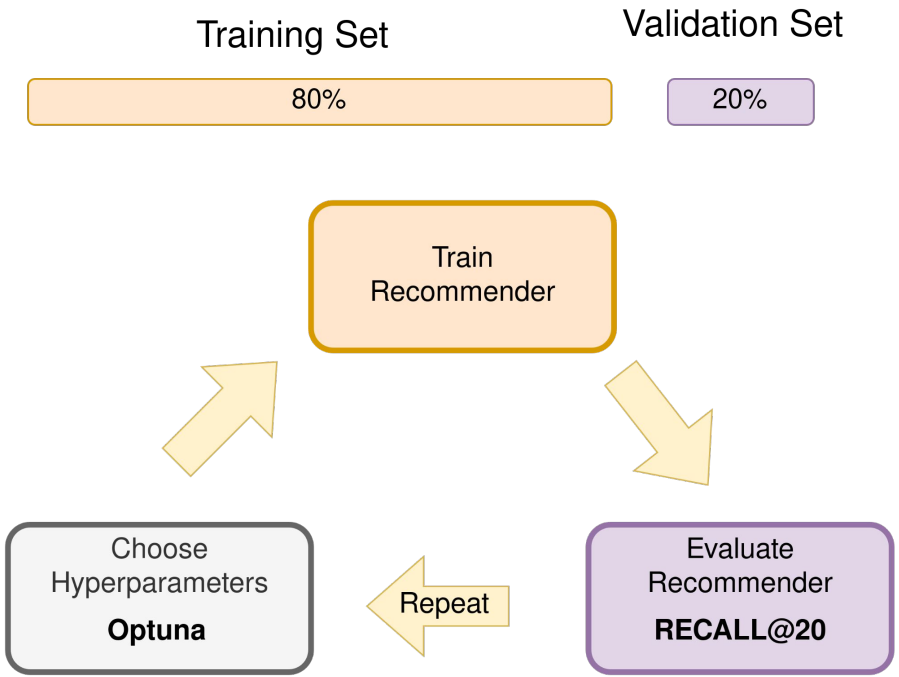
Holdout split

Hyper Parameters tuning with Optuna

Slim Elastic Net

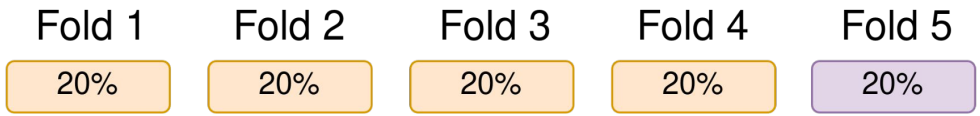
Public: 0.51462

Private: **0.51303**

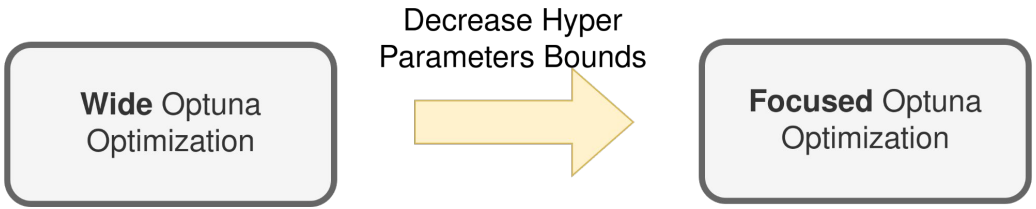


Cross Validation

5 Fold Cross Validation



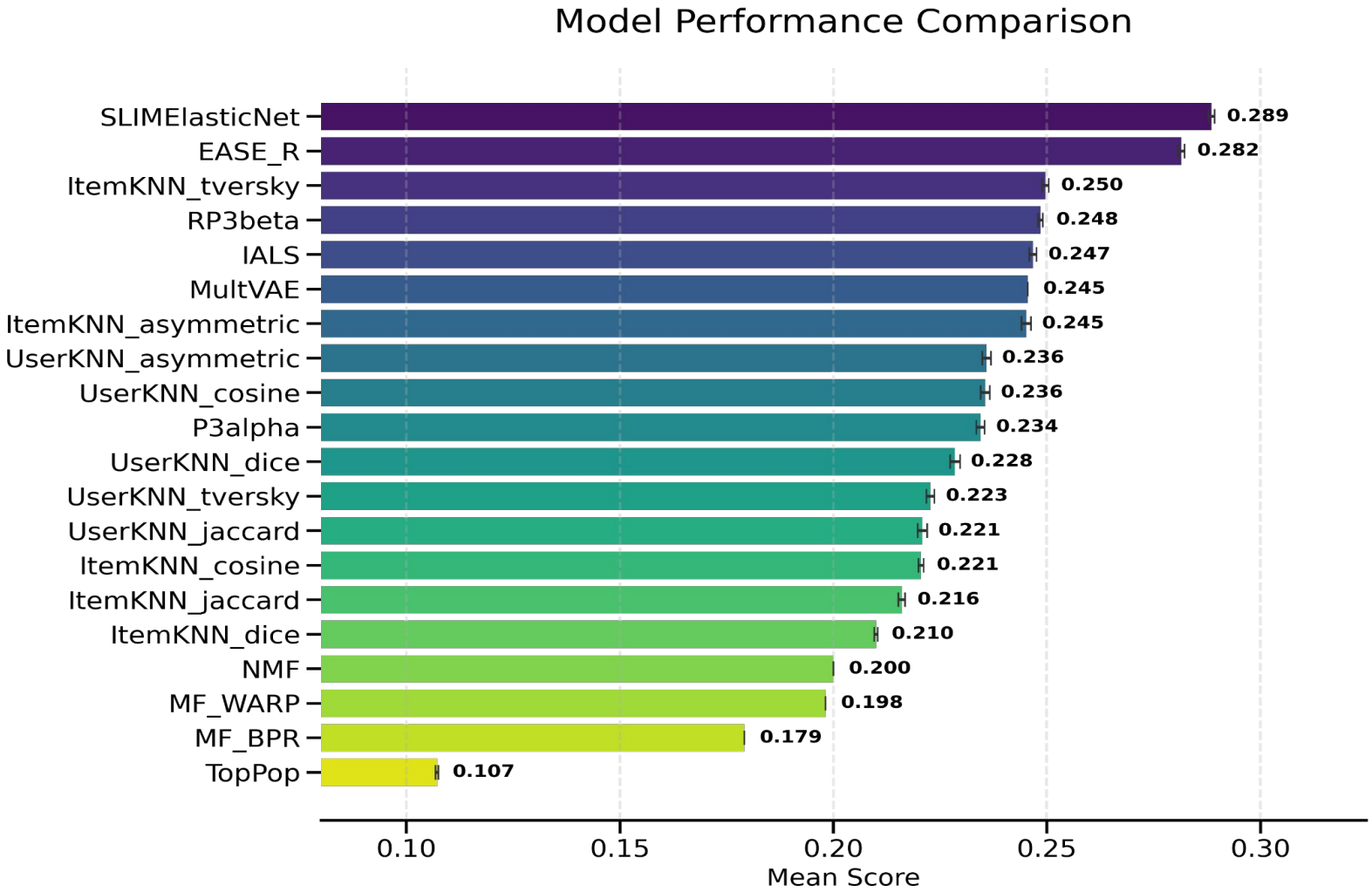
First a wider search then a more focused search



Slim Elastic Net

Public: 0.51514

Private: **0.51383**

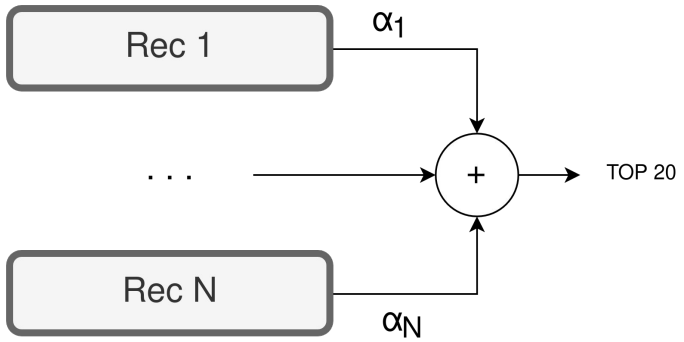

Model Performance Comparison
SLIMElasticNet
EASE_R
ItemKNN_tversky
RP3beta
IALS
MultVAE
ItemKNN_asymmetric
UserKNN_asymmetric
UserKNN_cosine
P3alpha
UserKNN_dice
UserKNN_tversky
UserKNN_jaccard
ItemKNN_cosine
ItemKNN_jaccard
ItemKNN_dice
NMF
MF_WARP
MF_BPR
TopPop
0.289
0.282
0.250
0.248
0.247
0.245
0.245
0.236
0.236
0.234
0.228
0.223
0.221
0.221
0.216
0.210
0.200
0.198
0.179
0.107
0.10
0.15
0.20
0.25
0.30
Mean Score

Hybrid Recommender

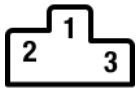
Score Blending

Similarity Matrices Addition
(SLIM+RP3 Beta)

Partition users by profile len

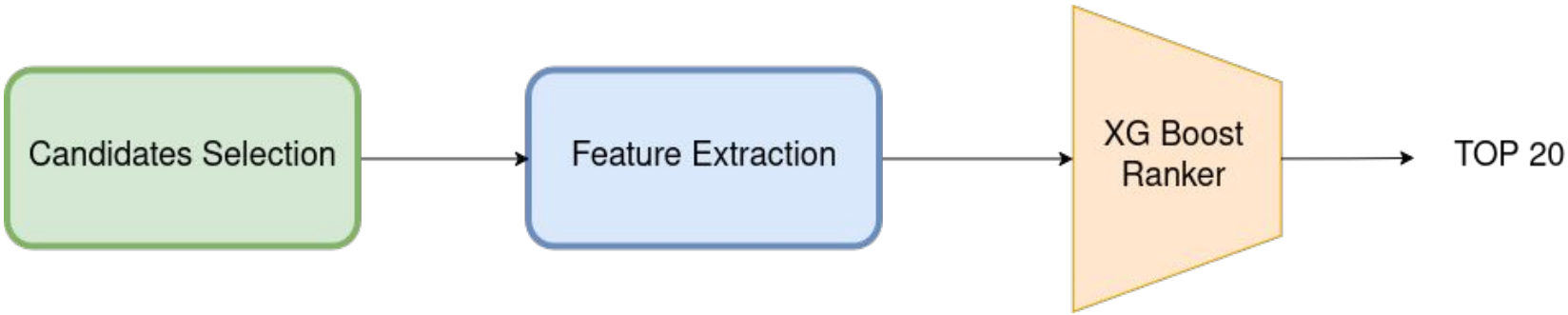


SLIM + IALS + Item KNN



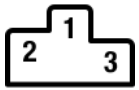
Public: 0.51524
Private: **0.51470**

XG Boost



First Attempt

- *Candidates: SLIM with higher cutoff*
- *Basic Features: Score, Pos, Recommended, User profile*
- *No XG Boost tuning*



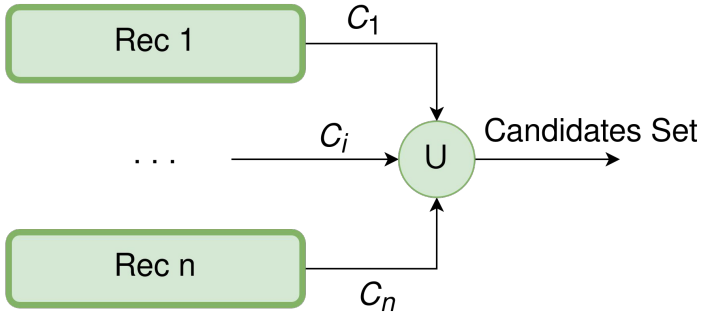
Public: 0.51974

Private: **0.51830**

Candidates Selection

The Baseline Approach:

- Select arbitrary cutoffs for a handful of models.
- Combine them using a simple union.
- *The Problem:* Blindly stacking models creates massive redundancy and floods the ranker with noise.



The Optimization Idea: Iteratively add batches of candidates that provide the highest number of *new* correct hits with the lowest amount of noise.

Greedy Selection via Marginal Utility

Maximize Marginal Precision $$maximize \frac{\Delta G_{m,c}}{\Delta C_{m,c}}$$

Marginal Gain (New Correct Hits): $$\Delta G_{m,c} = |(P_{m,c} \setminus S) \cap V|$$

Marginal Cost (New Items Added): $$\Delta C_{m,c} = |P_{m,c} \setminus S|$$

Where:

m : Base model

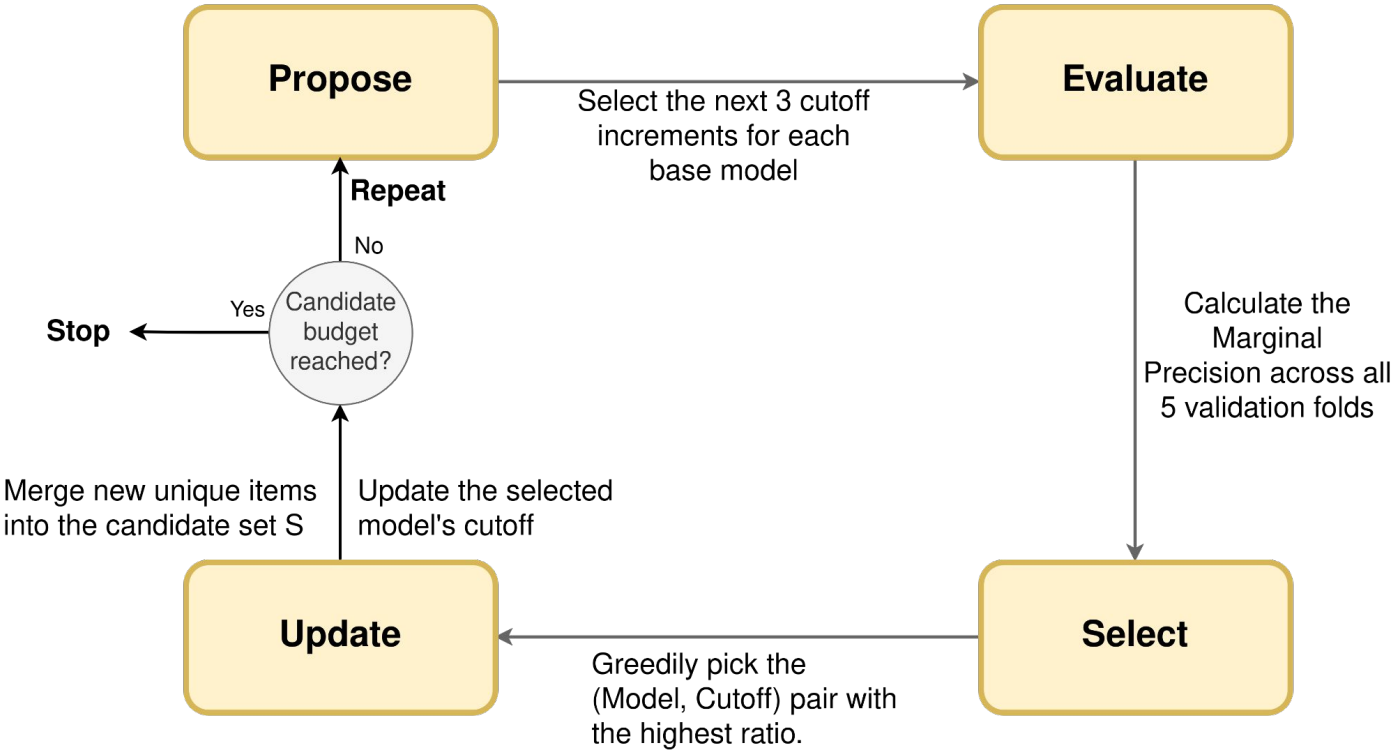
c : Proposed cutoff length

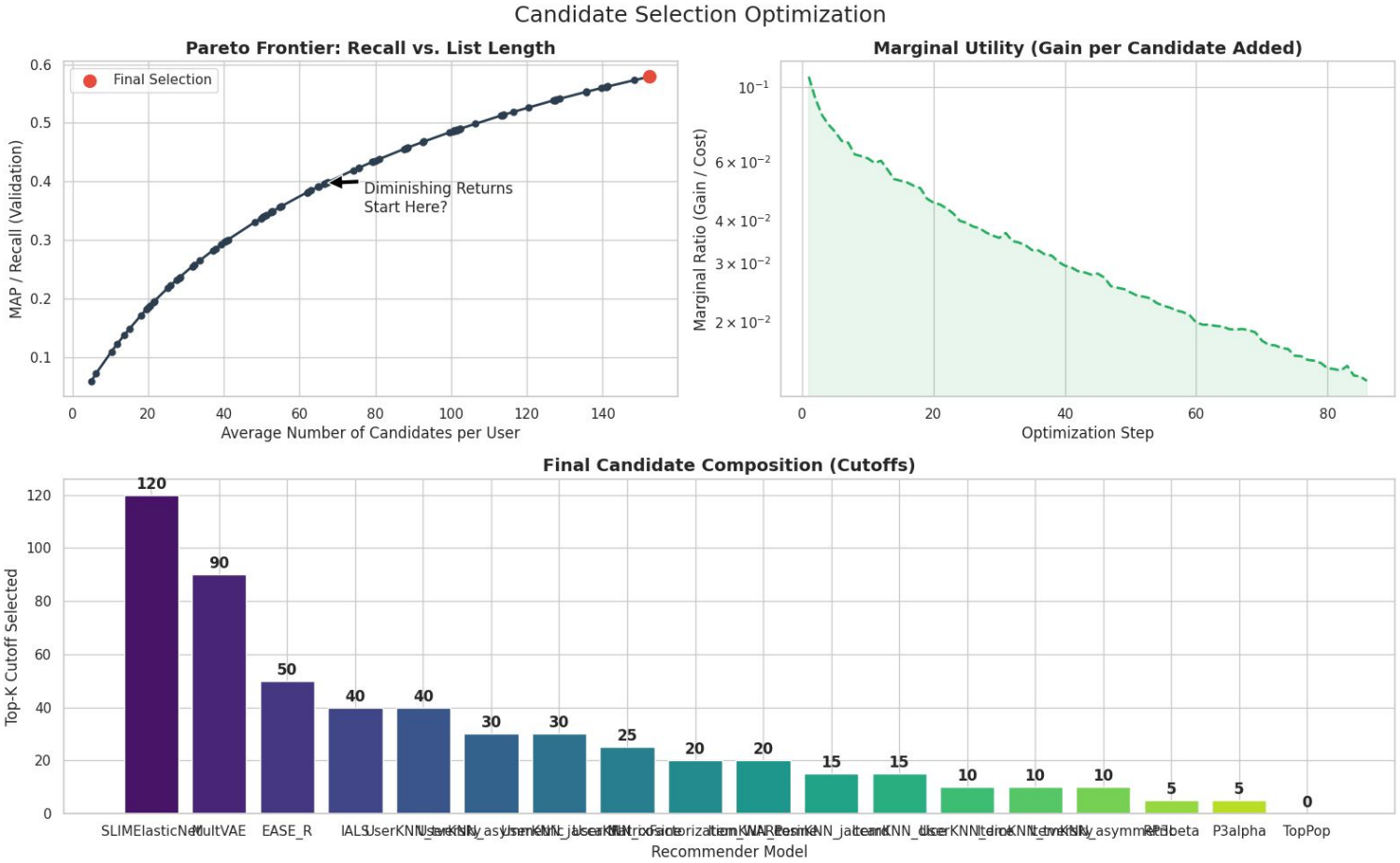
S : Current cumulative set of selected candidates

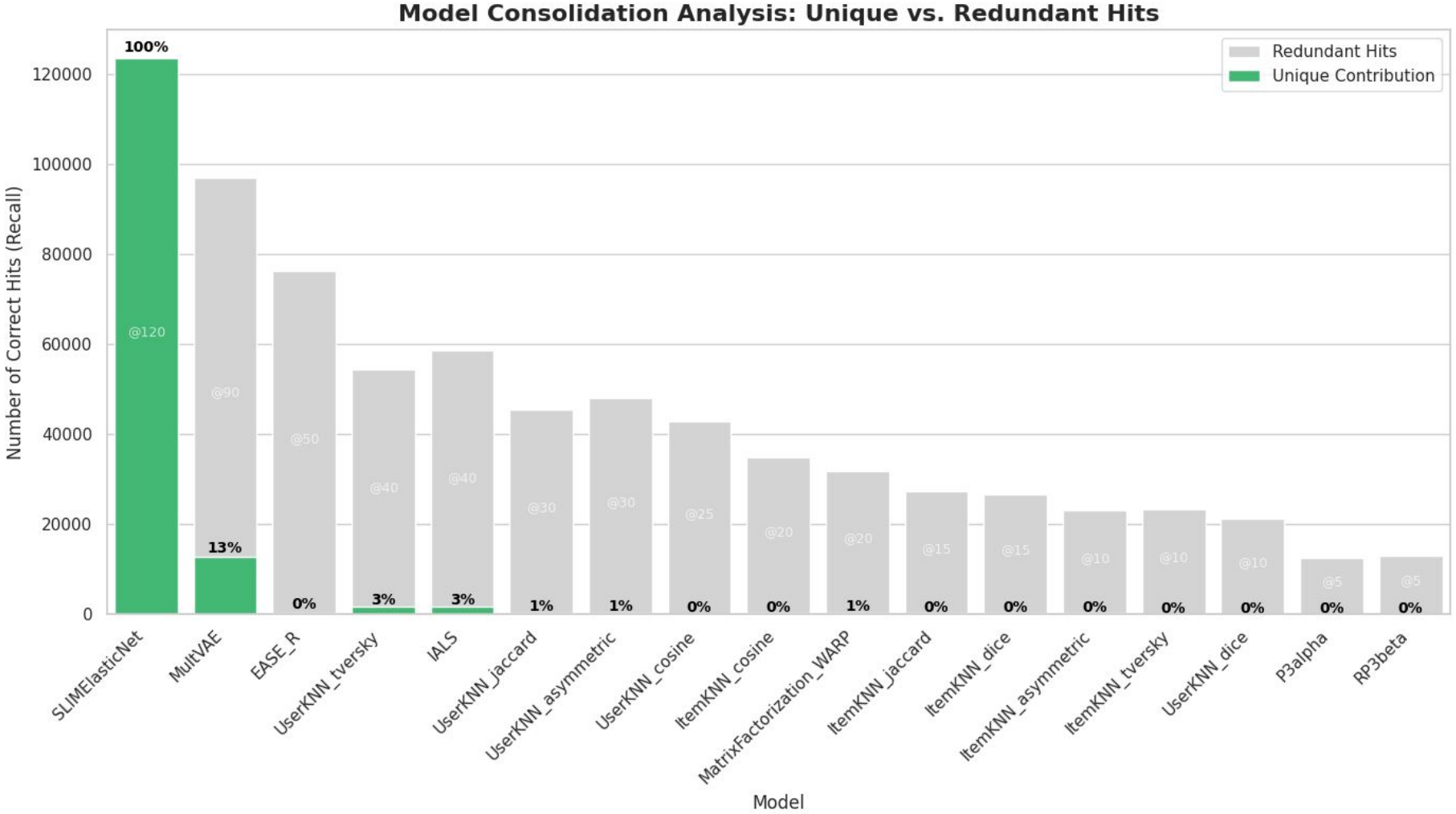
$P_{m,c}$: Proposed items from model m at cutoff c

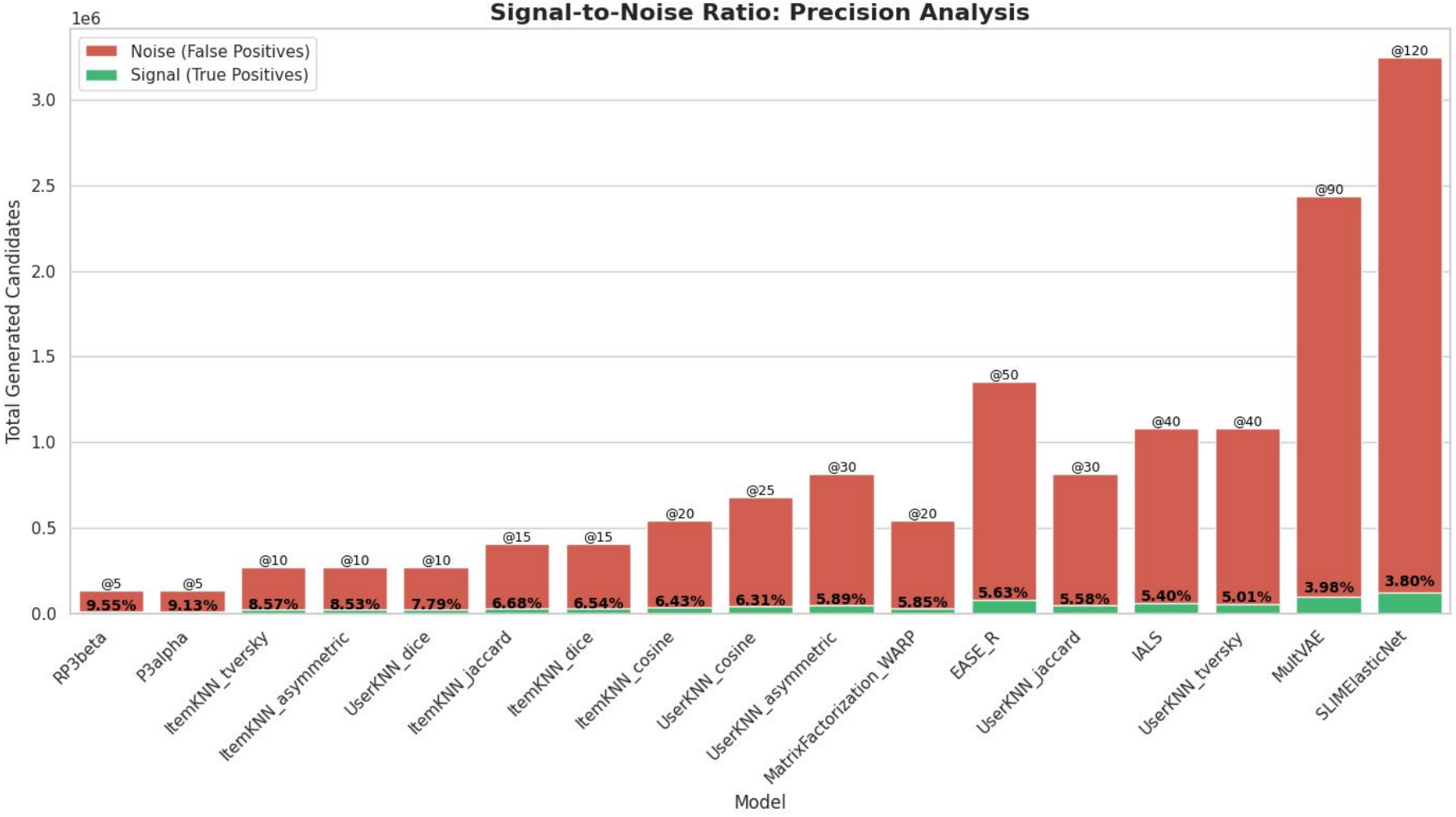
V : Items in the Validation set (Ground Truth)

Loop

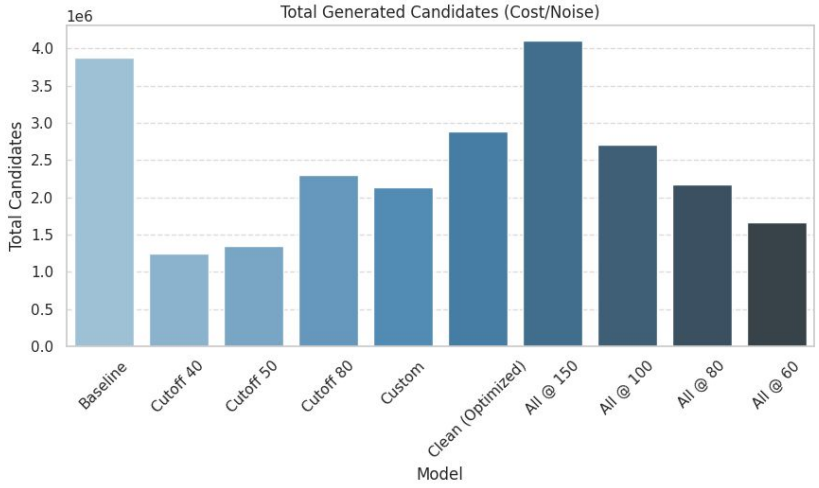

Propose
Select the next 3 cutoff increments for each base model
Evaluate
Repeat
No
Yes
Stop
Candidate budget reached?
Calculate the Marginal Precision across all 5 validation folds
Merge new unique items into the candidate set S
Update the selected model's cutoff
Update
Greedily pick the (Model, Cutoff) pair with the highest ratio.
Select

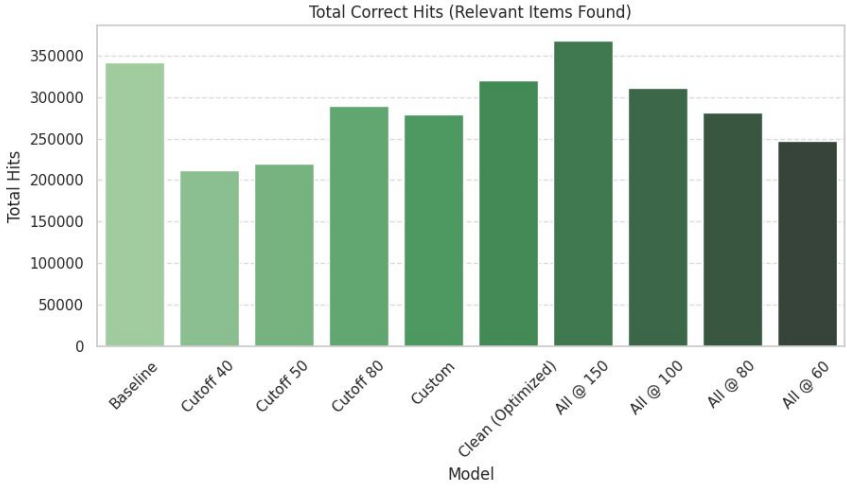

Candidate Selection Optimization
Pareto Frontier: Recall vs. List Length
Final Selection
Diminishing Returns
Start Here?
MAP / Recall (Validation)
Average Number of Candidates per User
Marginal Utility (Gain per Candidate Added)
Marginal Ratio (Gain / Cost)
Optimization Step
Final Candidate Composition (Cutoffs)
Top-K Cutoff Selected
Recommender Model
120
90
50
40
40
30
30
25
20
20
15
15
10
10
10
5
5
0
SLIM
ElasticNet
MultVAE
EASE_R
IALS
P3alpha
TopPop

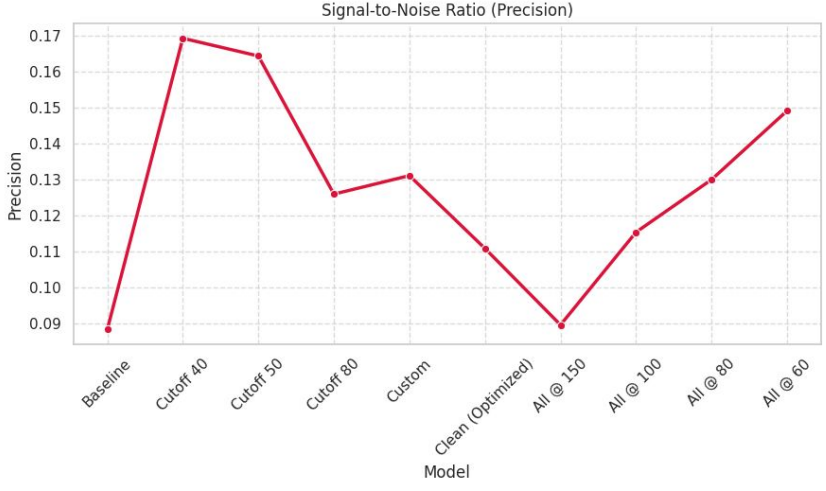

Model Consolidation Analysis: Unique vs. Redundant Hits
Redundant Hits
Unique Contribution
Number of Correct Hits (Recall)
0
20000
40000
60000
80000
100000
120000
100%
@120
13%
@90
0%
@50
3%
@40
3%
@40
1%
@30
1%
@30
0%
@25
0%
@20
1%
@20
0%
@15
0%
@15
0%
@10
0%
@10
0%
@10
0%
@5
0%
@5
SLIMElasticNet
MultVAE
EASE_R
UserKNN_tversky
IALS
UserKNN_jaccard
UserKNN_asymmetric
UserKNN_cosine
ItemKNN_cosine
MatrixFactorization_WARP
ItemKNN_jaccard
ItemKNN_dice
ItemKNN_asymmetric
ItemKNN_tversky
UserKNN_dice
P3alpha
RP3beta
Model

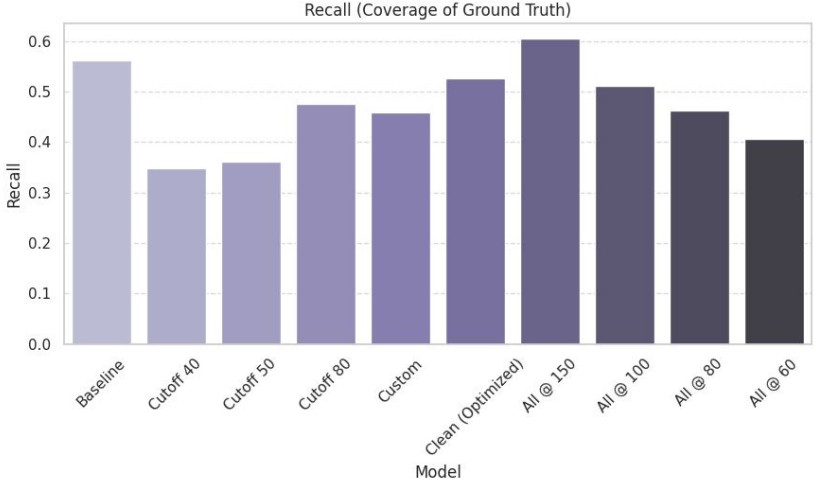

Signal-to-Noise Ratio: Precision Analysis
1e6
Noise (False Positives)
Signal (True Positives)
Total Generated Candidates
3.0
2.5
2.0
1.5
1.0
0.5
0.0
@5
9.55%
@5
9.13%
@10
8.57%
@10
8.53%
@10
7.79%
@15
6.68%
@15
6.54%
@20
6.43%
@25
6.31%
@30
5.89%
@20
5.85%
@50
5.63%
@30
5.58%
@40
5.40%
@40
5.01%
@90
3.98%
@120
3.80%
RP3beta
P3alpha
ItemKNN_tversky
ItemKNN_asymmetric
UserKNN_dice
ItemKNN_jaccard
ItemKNN_dice
ItemKNN_cosine
UserKNN_cosine
UserKNN_asymmetric
MatrixFactorization_WARP
EASE_R
UserKNN_jaccard
IALS
UserKNN_tversky
MultVAE
SLIMElasticNet
Model

Candidate Selection Performance Analysis





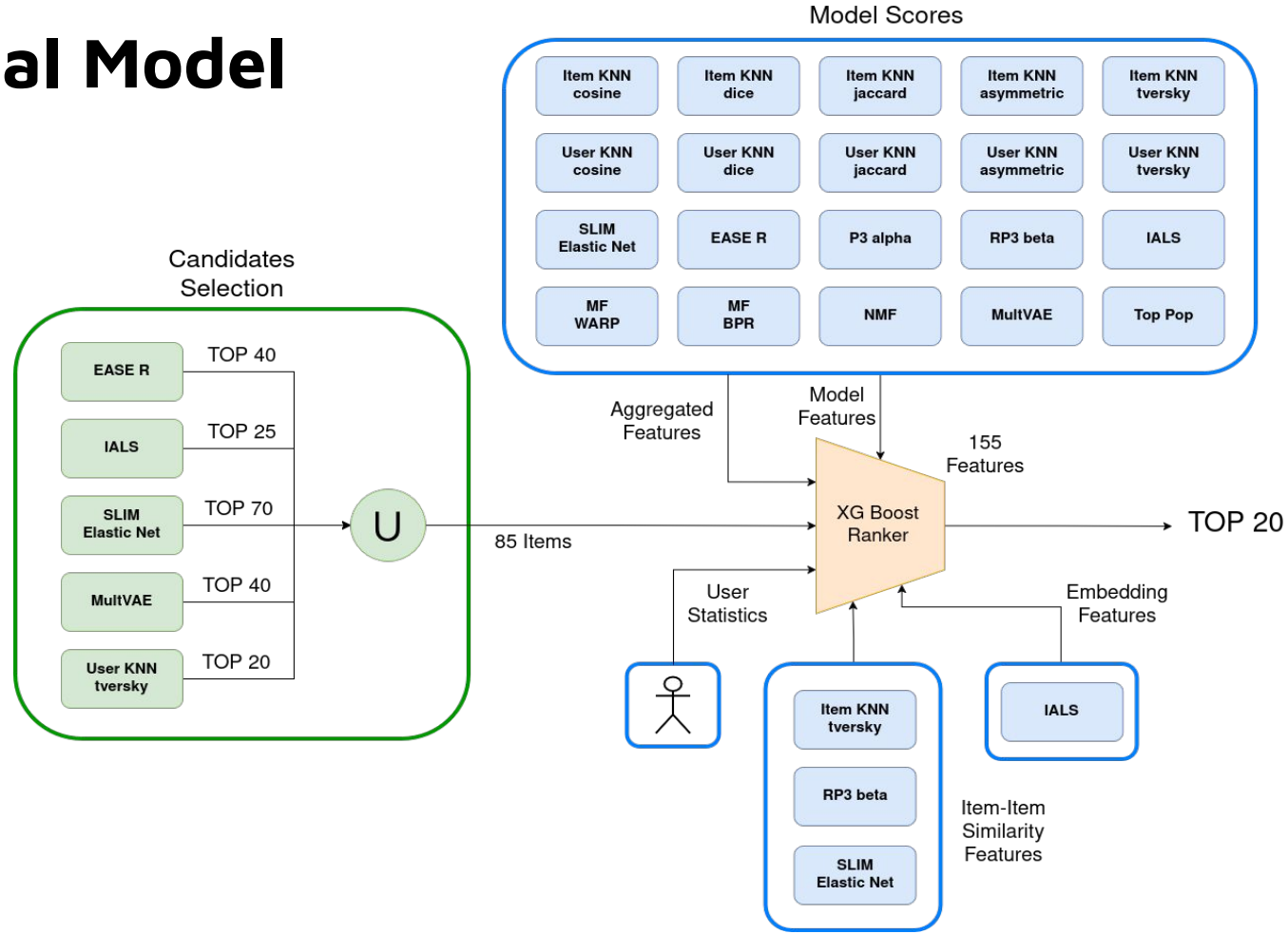


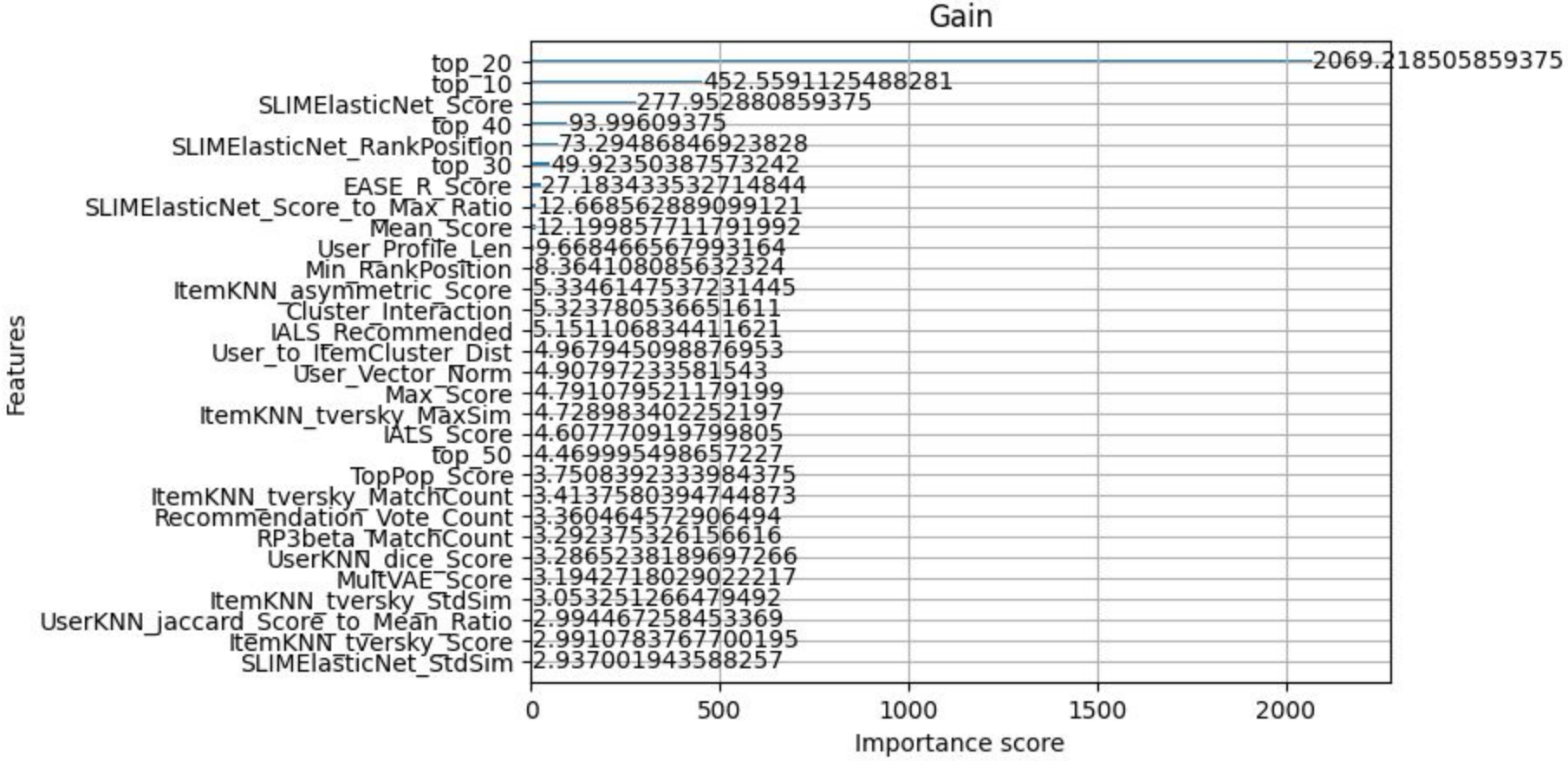


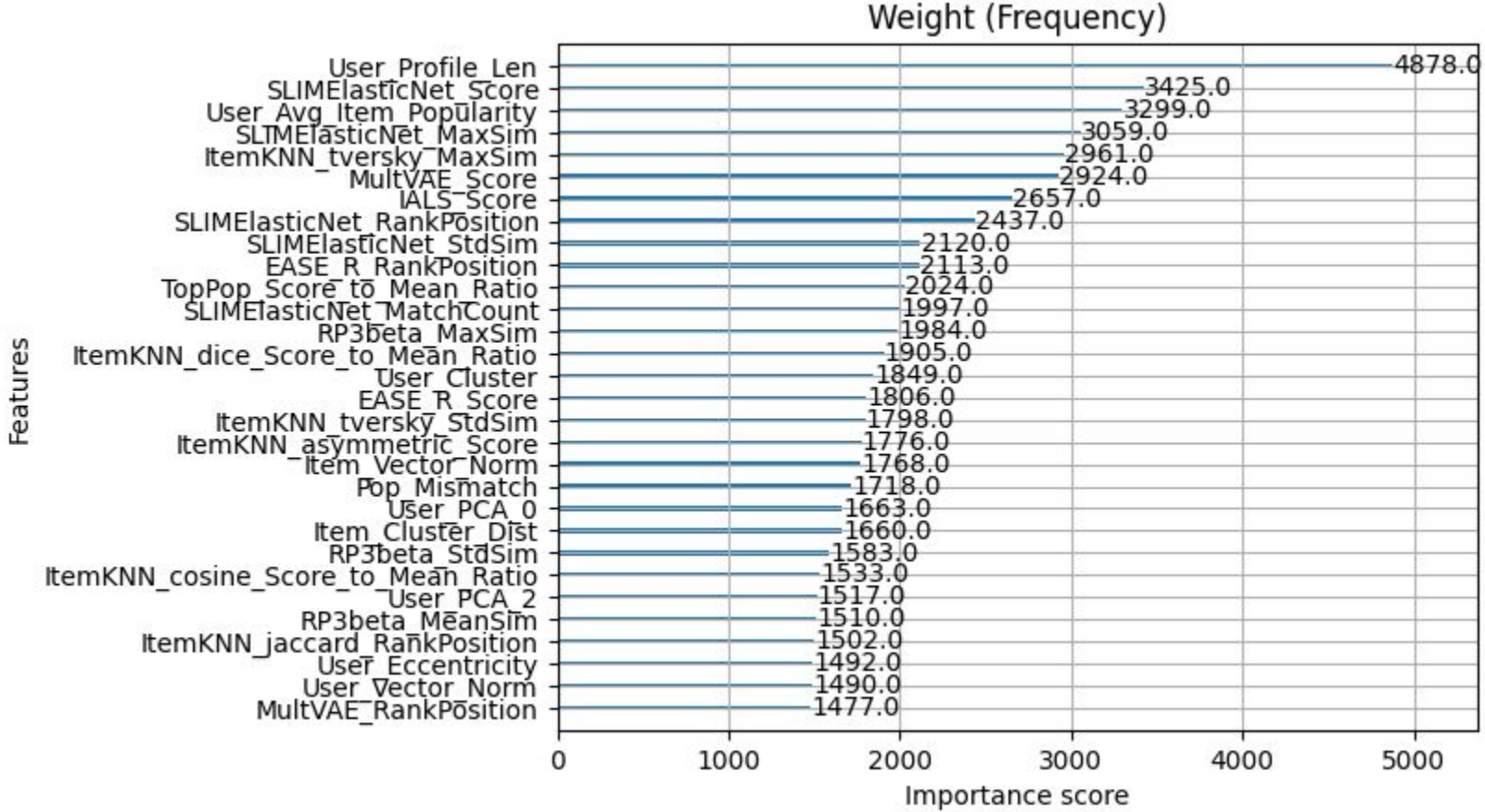
Feature Extraction

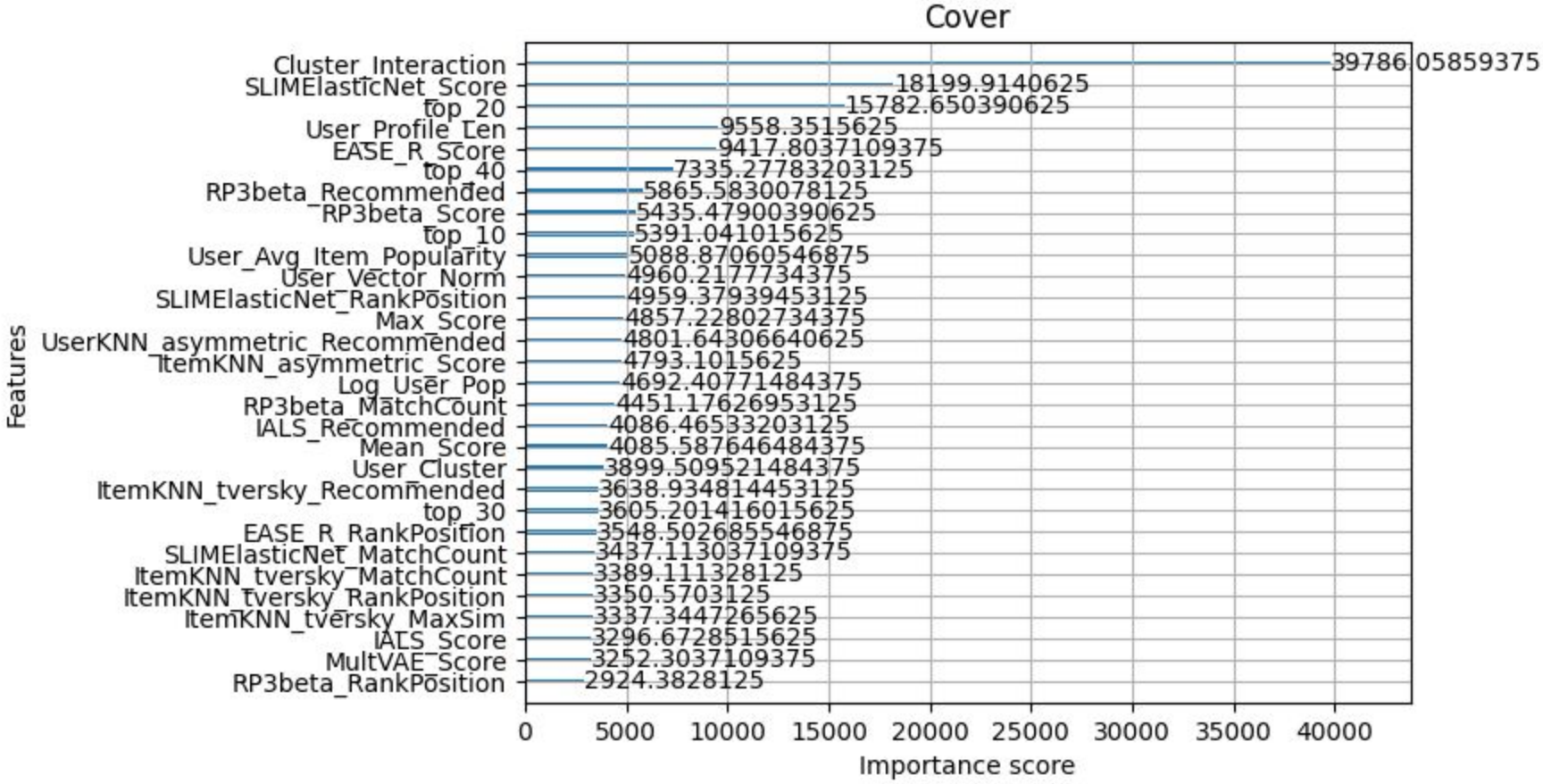
- **Candidates generation flags:** *top_10*, *top_20*, *top_30*, *top_40*, *top_50*
- **Models Features:** *Score*, *RankPosition*, *Recommended*
- **Item-Item Similarities features:** *Slim-Item KNN-RP3 Beta item-item similarities*
- **Embedding features:** *User clustering*, *Item clustering*, *PCA components*...
- **Aggregate Features:** *Max*, *Min*, *Mean*, *Std of features*
- **User Statistics:** *User profile len*, *Item popularity*, *ratio*...

Final Model


Model Scores
Item KNN cosine
Item KNN dice
Item KNN jaccard
Item KNN asymmetric
Item KNN tversky
User KNN cosine
User KNN dice
User KNN jaccard
User KNN asymmetric
User KNN tversky
SLIM Elastic Net
EASE R
P3 alpha
RP3 beta
IALS
MF WARP
MF BPR
NMF
MultVAE
Top Pop
Candidates Selection
EASE R
TOP 40
IALS
TOP 25
SLIM Elastic Net
TOP 70
MultVAE
TOP 40
User KNN tversky
TOP 20
U
85 Items
Aggregated Features
Model Features
155 Features
XG Boost Ranker
TOP 20
User Statistics
Embedding Features
Item KNN tversky
RP3 beta
SLIM Elastic Net
IALS
Item-Item Similarity Features


Gain
top_20
top_10
SLIMElasticNet_Score
top_40
SLIMElasticNet_RankPosition
top_30
EASE_R_Score
SLIMElasticNet_Score_to_Max_Ratio
Mean_Score
User_Profile_Len
Min_RankPosition
ItemKNN_asymmetric_Score
Cluster_Interaction
IALS_Recommended
User_to_ItemCluster_Dist
User_Vector_Norm
Max_Score
ItemKNN_tversky_MaxSim
IALS_Score
top_50
TopPop_Score
ItemKNN_tversky_MatchCount
Recommendation_Vote_Count
RP3beta_MatchCount
UserKNN_dice_Score
MultVAE_Score
ItemKNN_tversky_StdSim
UserKNN_jaccard_Score_to_Mean_Ratio
ItemKNN_tversky_Score
SLIMElasticNet_StdSim
2069.218505859375
452.5591125488281
277.952880859375
93.99609375
73.29486846923828
49.92350387573242
27.183433532714844
12.668562889099121
12.199857711791992
9.668466567993164
8.3641080856323 24
5.3346147537231445
5.323780536651611
5.151106834411621
4.967945098876953
4.90797233581543
4.791079521179199
4.728983402252197
4.607770919799805
4.469995498657227
3.7508392333984375
3.4137580394744873
3.360464572906494
3.292375326156616
3.2865238189697266
3.1942718029022217
3.053251266479492
2.994467258453369
2.9910783767700195
2.937001943588257
Features
0
500
1000
1500
2000
Importance score


Weight (Frequency)
User_Profile_Len 4878.0
SLIMElasticNet_Score 3425.0
User_Avg_Item_Popularity 3299.0
SLIMElasticNet_MaxSim 3059.0
ItemKNN_tversky_MaxSim 2961.0
MultVAE_Score 2924.0
IALS_Score 2657.0
SLIMElasticNet_RankPosition 2437.0
SLIMElasticNet_StdSim 2120.0
EASE_R_RankPosition 2113.0
TopPop_Score_to_Mean_Ratio 2024.0
SLIMElasticNet_MatchCount 1997.0
RP3beta_MaxSim 1984.0
ItemKNN_dice_Score_to_Mean_Ratio 1905.0
User_Cluster 1849.0
EASE_R_Score 1806.0
ItemKNN_tversky_StdSim 1798.0
ItemKNN_asymmetric_Score 1776.0
Item_Vector_Norm 1768.0
Pop_Mismatch 1718.0
User_PCA_0 1663.0
Item_Cluster_Dist 1660.0
RP3beta_StdSim 1583.0
ItemKNN_cosine_Score_to_Mean_Ratio 1533.0
User_PCA_2 1517.0
RP3beta_MeanSim 1510.0
ItemKNN_jaccard_RankPosition 1502.0
User_Eccentricity 1492.0
User_Vector_Norm 1490.0
MultVAE_RankPosition 1477.0
Features
0
1000
2000
3000
4000
5000
Importance score

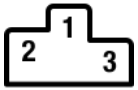

Cover
Cluster_Interaction
SLIMElasticNet_Score
top_20
User_Profile_Len
EASE_R_Score
top_40
RP3beta_Recommended
RP3beta_Score
top_10
User_Avg_Item_Popularity
User_Vector_Norm
SLIMElasticNet_RankPosition
Max_Score
UserKNN_asymmetric_Recommended
ItemKNN_asymmetric_Score
Log_User_Pop
RP3beta_MatchCount
IALS_Recommended
Mean_Score
User_Cluster
ItemKNN_tversky_Recommended
top_30
EASE_R_RankPosition
SLIMElasticNet_MatchCount
ItemKNN_tversky_MatchCount
ItemKNN_tversky_RankPosition
ItemKNN_tversky_MaxSim
IALS_Score
MultVAE_Score
RP3beta_RankPosition
39786.05859375
18199.9140625
15782.650390625
9558.3515625
9417.8037109375
7335.27783203125
5865.5830078125
5435.47900390625
5391.041015625
5088.87060546875
4960.2177734375
4959.37939453125
4857.22802734375
4801.64306640625
4793.1015625
4692.40771484375
4451.17626953125
4086.46533203125
4085.587646484375
3899.509521484375
3638.934814453125
3605.201416015625
3548.502685546875
3437.113037109375
3389.111328125
3350.5703125
3337.3447265625
3296.6728515625
3252.3037109375
2924.3828125
Features
0
5000
10000
15000
20000
25000
30000
35000
40000
Importance score

Results

- **Out-of-Fold (OOF)** predictions for XG Boost training dataframe

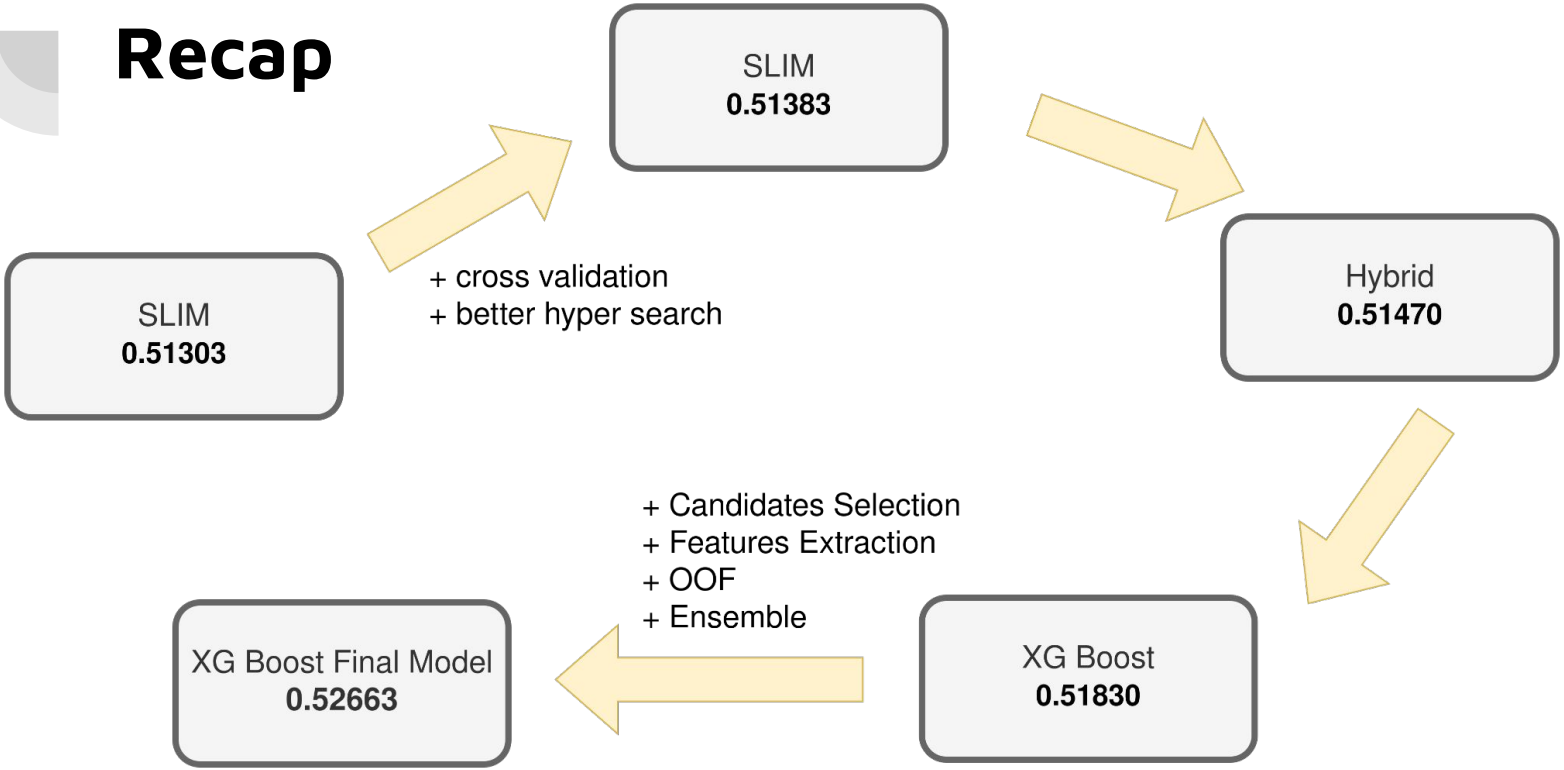
- **Ensemble** of 5 XGBoost models, leveraging different random seeds and slightly varied hyperparameters.

XG Boost Final Model



Public: 0.52806
Private: **0.52663**

Recap


SLIM
0.51303
+ cross validation
+ better hyper search
SLIM
0.51383
Hybrid
0.51470
XG Boost
0.51830
+ Candidates Selection
+ Features Extraction
+ OOF
+ Ensemble
XG Boost Final Model
0.52663

Future Improvements

Looking back, several areas could be improved for future iterations:

- **Model Diversity:** Integrate new models (e.g. LightGCN)
- **Hybridization:** Deeper exploration of early-stage hybrids.
- **Candidates Generation**: Evaluation of more aggressive filtering. Less candidates means less noise and faster training times.
- **OOF or Cross Validation:** Re-evaluate OOF vs. standard Cross-Validation.
- **Ensembling Strategy:** Combine structurally distinct models (beyond random seeds).